CS 527 Research Project Report: Group-29

Bangji Yang, Yuhan Ding
University of Illinois at Urbana-Champaign
IL, USA
[bangjiy2,yuhand7]@illinois.edu

Abstract

Automated program repair with Large Language Models (LLMs) often struggles with the "Contextual Dilemma": insufficient context leads to diagnostic failure, while excessive context introduces noise and escalates costs. This project proposes **Ada-MSS**, an **Adaptive Multi-granularity Semantic Slicing** framework that dynamically adjusts the context window based on execution feedback. Unlike static pruning, Ada-MSS employs an iterative "Zoom-In" strategy, starting from a trace-anchored atomic core and progressively expanding to higher semantic layers—such as pruned syntax skeletons and cross-functional dependency slices—only when initial repair attempts fail. By leveraging runtime feedback as a heuristic signal, we aim to identify the *Minimal Sufficient Context* for diverse bug categories, effectively maximizing repair accuracy while minimizing token overhead.

1 Artifact Information

Link to the GitHub Repository: https://github.com/Islet-Yang/Ada-MSS

2 Introduction

Automated software debugging is essential for maintaining large-scale systems, yet the efficiency of LLM-based repair is heavily gated by context management. Currently, the industry relies on "maximalist" strategies that feed entire files into prompts, which is both noisy and expensive[9].

State-of-the-art techniques, such as SWE-agent[10], attempt to solve this via multi-step file system exploration. However, these methods are often "blind" to the granularity of code they ingest, leading to the *"Lost in the Middle"* [7]effect where critical fault logic is buried under thousands of lines of irrelevant documentation and boilerplate code.

A significant research gap exists in **dynamic context optimization**: existing tools do not distinguish between bugs that require only local fixes and those necessitating broader structural understanding[5]. This results in an "all-or-nothing" approach that lacks the nuance of human developers, who typically start with a specific error line and only expand their scope when the local context proves insufficient.

We propose **Ada-MSS** to bridge this gap. Our technique introduces a feedback-driven loop that treats context as a dynamic variable rather than a fixed input. By initiating the repair process with a **Trace-Anchored Core (TAC)** and conditionally escalating to **Pruned Syntax Skeletons (PSS)** or **Cross-functional Dependency Slices (CDS)** based on unit test results, Ada-MSS mimics the human cognitive process of hierarchical debugging.

Our experimental plan focuses on evaluating Ada-MSS using the **DebugBench**[8] dataset. We will compare our adaptive strategy against static baselines across GPT-4o-mini[3] and DeepSeek-V3[6], measuring not just the final repair success rate, but the **Token-to-Success (TTS)** efficiency.

We anticipate that Ada-MSS will achieve parity with monolithic baselines in repair accuracy while achieving a multi-fold reduction in cumulative token usage. This would prove that execution feedback is a potent heuristic for context pruning.

Our primary contributions include:

- The **Ada-MSS framework**, a feedback-loop-driven adaptive context manager.
- An investigation into **Context Escalation Policies**, identifying which bug types trigger higher granularity levels.
- A comparative study of LLM performance under dynamic "Semantic Zooming."

3 Proposed Technique

3.1 Approach Overview

The Ada-MSS framework operates as an iterative feedback loop between a **Semantic Pruning Engine** and a **Validation Sandbox**. As depicted in Figure 1, the workflow is not a linear pipeline but a recursive cycle.

Initially, the engine generates the most concise representation (TAC). If the LLM's proposed patch fails the validation tests, the system triggers a "Context Escalation" signal. The engine then re-parses the AST to provide the next granularity level (PSS or CDS). This cycle repeats until either a valid patch is found or the maximum context (UMC) is reached. This "Semantic Zooming" ensures that the model only consumes as much information as is strictly necessary for the specific complexity of the bug.

1. Context Escalation Policy: The heart of our adaptive approach is the escalation logic. Unlike simple retries, our policy uses the *Error Type* from the validation failure to decide how much to expand. For example, a `NameError` might trigger a PSS expansion to show variable definitions, while an `AttributeError` might skip directly to CDS to reveal class signatures.

2. AST-Based Iterative Slicer: The slicer is stateful; it keeps track of the current "abstraction radius" and expands the tree-traversal depth upon receiving an escalation signal. It uses Python's `ast` module to ensure that each expanded slice remains syntactically valid and logically coherent.

4 Experimental Evaluation

In this project, we aim to answer the following research questions (RQs) to evaluate the effectiveness and efficiency of our Ada-MSS framework:

- **RQ1 (Effectiveness):** Can the adaptive semantic slicing strategy achieve repair accuracy comparable to the unconstrained monolithic context (UMC) baseline?

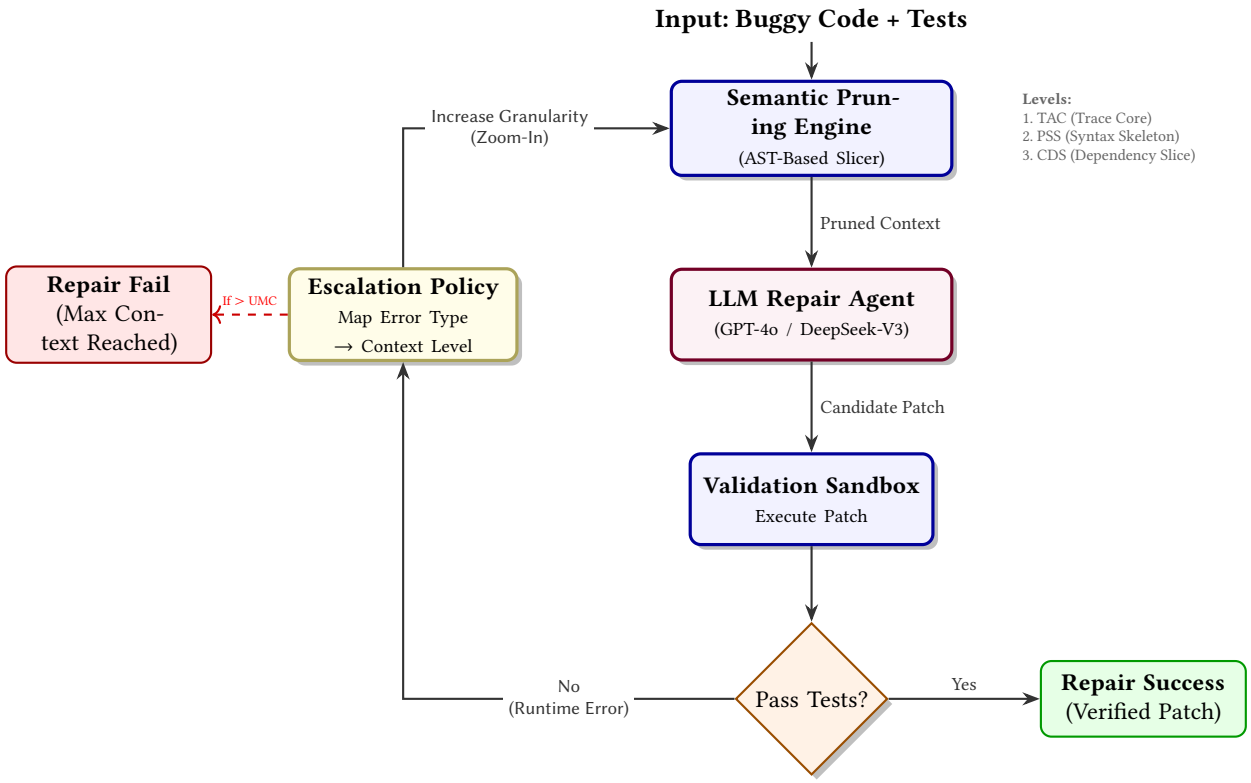

Figure 1: Overview of the Ada-MSS Framework

- **RQ2 (Efficiency):** To what extent does the feedback-driven "Zoom-In" approach reduce cumulative token consumption compared to static, full-context prompting?
- **RQ3 (Adaptivity Analysis):** How does the error type from execution feedback influence the context escalation depth, and what is the success rate at each granularity level (TAC, PSS, CDS)?
- **RQ4 (Generalizability):** Does the performance gain of Ada-MSS remain consistent across different LLM architectures (e.g., GPT-4o-mini vs. DeepSeek-V3)?

4.1 Evaluation Setting

Dataset: We will conduct our experiments on the **DebugBench** dataset, specifically focusing on the Python subset. DebugBench provides a diverse set of bugs across various categories such as logical errors, API misuse, and structural flaws. Crucially, each instance includes runtime test cases, which are essential for driving our iterative feedback loop.

Baselines: To assess the performance of Ada-MSS, we compare it against two primary baselines:

- **Static UMC (Upper Bound):** Feeding the entire source file into the LLM, representing the traditional "maximalist" context approach.
- **Static TAC (Lower Bound):** Providing only the error-anchored line, representing the minimum possible context.

Models and Implementation: We will use **GPT-4o-mini** and **DeepSeek-V3** via their respective APIs. The *Semantic Pruning Engine* will be implemented using Python's `ast` library to ensure precise code slicing. The *Validation Sandbox* will execute the proposed patches within a secure container to prevent side effects and capture detailed traceback logs for escalation.

Metrics:

- **Pass@1:** The percentage of bugs fixed that pass all test cases within the maximum escalation limit.
- **Token-to-Success (TTS):** The average number of tokens consumed per successful fix.
- **Escalation Rate:** The frequency with which the system moves from TAC to higher semantic layers.

4.2 Evaluation Results

Present the results of individual research questions in this subsection. Use tables/figures to present the results. You need to discuss discuss your observations and speculate the reasons for observations.

5 Relate Work

LLM-based Automated Program Repair (APR): The advent of Large Language Models (LLMs) has significantly advanced the field of APR. Benchmarks such as *HumanEval-Fix*[2] and the repository-level *SWE-bench* [4] have established new standards for evaluating LLMs on real-world software issues. While state-of-the-art agents

like *SWE-agent*[10] demonstrate strong capabilities in navigating complex codebases, they typically rely on "maximalist" context injection or iterative file exploration. Our work, *Ada-MSS*, shifts the focus towards optimizing the *granularity* of the input, seeking to minimize token overhead through semantic pruning.

Fault Localization and Program Slicing: Traditional Fault Localization (FL)[1] techniques, such as Spectrum-based FL (SBFL), analyze execution traces to identify suspicious code regions. Recent research has explored using LLMs as neural localizers to interpret stack traces directly. However, these methods often assume a static context window. We build upon the concepts of traditional *program slicing* but adapt them for neural reasoning by introducing a feedback-driven escalation loop. By adjusting the "abstraction radius" based on runtime feedback, we investigate how structural code layers influence the model's diagnostic intuition.

Context Efficiency and Information Noise: The *"Lost in the Middle"* phenomenon [7] highlights a critical limitation: LLMs' reasoning performance often degrades when presented with excessive, irrelevant information. In the software engineering context, this manifests as "contextual noise" from boilerplate code and documentation. While retrieval-augmented approaches like *RepoCoder*[11] attempt to prune context via RAG, they often lack awareness of the code's semantic structure. *Ada-MSS* addresses this by using AST-based slicing and a dynamic "Semantic Zooming" policy to ensure that only the most relevant logical dependencies are preserved.

References

[1] Rui Abreu, Peter Zoeteweij, and Arjan JC Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *Testing: Academic and industrial conference practice and research techniques-MUTATION (TAICPART-MUTATION 2007)*. IEEE, 89–98.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[3] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).

[4] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).

[5] Yingling Li, Muxin Cai, Junjie Chen, Yang Xu, Lei Huang, and Jianping Li. 2025. Context-aware prompting for LLM-based program repair: Y. Li et al. *Automated Software Engineering* 32, 2 (2025), 42.

[6] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[7] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* 12 (2024), 157–173.

[8] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Hui Haotian, Liu Weichuan, Zhiyuan Liu, et al. 2024. Debugbench: Evaluating debugging capability of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*. 4173–4198.

[9] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective retrieval for repository-level code completion. *arXiv preprint arXiv:2403.10059* (2024).

[10] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[11] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484.